

# Expense Tracker

## Project Description:

Build an Expense Tracker where users can record their income and expenses. The website should show the total income, total expenses, and the remaining balance. Users should also be able to view and manage their transaction history.

## Features:

- Add Income
- Add Expense
- Categories
- Balance
- Monthly Report
- Recent Transactions

## Member 1

- Dashboard
- Summary Cards

## Member 2

- Income Module

## Member 3

- Expense Module
- Categories
- Filters

## Member 4

- Reports
- Local Storage
- Charts (Bonus)

## Member 1: Dashboard & Summary Cards

Member 1 is responsible for the top summary cards (Balance, Total Income, Total Expenses) and calculating the overall totals.

### HTML (member1.html)

```
<div class="dashboard-grid">
  <div class="card balance">
    <h3>Remaining Balance</h3>
    <div class="amount" id="total-balance">$0.00</div>
  </div>
  <div class="card income">
    <h3>Total Income</h3>
    <div class="amount" id="total-income">$0.00</div>
  </div>
  <div class="card expense">
    <h3>Total Expenses</h3>
    <div class="amount" id="total-expenses">$0.00</div>
  </div>
</div>
```

### CSS (member1.css)

```
.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.5rem;
  margin-bottom: 2rem;
}

.card {
  background: #ffffff;
  padding: 1.5rem;
  border-radius: 12px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  transition: transform 0.2s;
}

.card:hover { transform: translateY(-2px); }

.card h3 {
  font-size: 0.9rem;
  text-transform: uppercase;
  color: #6b7280;
  margin-bottom: 0.5rem;
}

.card .amount {
  font-size: 1.8rem;
  font-weight: bold;
}

.card.balance .amount { color: #4f46e5; }
.card.income .amount { color: #10b981; }
.card.expense .amount { color: #ef4444; }
```

## JavaScript (member1.js)

### JavaScript

```
function updateDashboard(filteredTransactions) {
  let totalIncome = 0;
  let totalExpense = 0;

  filteredTransactions.forEach(tx => {
    if (tx.type === 'income') totalIncome += tx.amount;
    if (tx.type === 'expense') totalExpense += tx.amount;
  });

  const remainingBalance = totalIncome - totalExpense;

  document.getElementById('total-income').innerText = `$$${totalIncome.toFixed(2)}`;
  document.getElementById('total-expenses').innerText = `$$${totalExpense.toFixed(2)}`;

  const balanceEl = document.getElementById('total-balance');
  balanceEl.innerText = `$$${remainingBalance.toFixed(2)}`;
  balanceEl.style.color = remainingBalance < 0 ? '#ef4444' : '#4f46e5';
}
```

## Member 2: Income Module

Member 2 handles the form input for logging new income sources.

### HTML (member2.html)

#### HTML

```
<div class="panel">
  <h2>Add Income</h2>
  <form id="income-form">
    <div class="form-group">
      <label for="inc-amount">Amount ($)</label>
      <input type="number" id="inc-amount" step="0.01" required min="0.01">
    </div>
    <div class="form-group">
      <label for="inc-category">Category</label>
      <select id="inc-category">
        <option value="Salary">Salary</option>
        <option value="Freelance">Freelance</option>
        <option value="Investments">Investments</option>
        <option value="Other Income">Other</option>
      </select>
    </div>
    <div class="form-group">
      <label for="inc-date">Date</label>
      <input type="date" id="inc-date" required>
    </div>
    <button type="submit" class="btn-income">Log Income</button>
  </form>
</div>
```

### CSS (member2.css)

```
.form-group {
  margin-bottom: 1rem;
}
label {
  display: block;
  margin-bottom: 0.5rem;
  font-weight: 500;
  font-size: 0.9rem;
}
input, select {
  width: 100%;
  padding: 0.75rem;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  font-size: 1rem;
}
button {
  width: 100%;
  padding: 0.75rem;
  border: none;
  border-radius: 6px;
  color: white;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
}
.btn-income { background-color: #10b981; }
.btn-income:hover { opacity: 0.9; }
```

---

### JavaScript (member2.js)

```
document.getElementById('inc-date').valueAsDate = new Date();

document.getElementById('income-form').addEventListener('submit', function(e) {
  e.preventDefault();
  const amount = parseFloat(document.getElementById('inc-amount').value);
  const category = document.getElementById('inc-category').value;
  const date = document.getElementById('inc-date').value;

  const incomeTx = {
    id: Date.now(),
    type: 'income',
    amount,
    category,
    date
  };

  transactions.push(incomeTx);
  saveToLocalStorage(); // Function maintained by Member 4
  this.reset();
  document.getElementById('inc-date').valueAsDate = new Date();
});
```

### Member 3: Expense Module, Categories, & Filters

Member 3 creates the Expense input form, category handling, and filters for viewing transaction history.

HTML (member3.html)

HTML

```
<div class="panel">
  <h2>Add Expense</h2>
  <form id="expense-form">
    <div class="form-group">
      <label for="exp-amount">Amount ($)</label>
      <input type="number" id="exp-amount" step="0.01" required min="0.01">
    </div>
    <div class="form-group">
      <label for="exp-category">Category</label>
      <select id="exp-category">
        <option value="Food & Dining">Food & Dining</option>
        <option value="Rent & Utilities">Rent & Utilities</option>
        <option value="Entertainment">Entertainment</option>
        <option value="Transportation">Transportation</option>
        <option value="Shopping">Shopping</option>
      </select>
    </div>
    <div class="form-group">
      <label for="exp-date">Date</label>
      <input type="date" id="exp-date" required>
    </div>
    <button type="submit" class="btn-expense">Log Expense</button>
  </form>
</div>

<div class="panel">
  <h2>Recent Transactions</h2>
  <div class="filters-container">
    <select id="filter-type">
      <option value="all">All Types</option>
      <option value="income">Income</option>
      <option value="expense">Expense</option>
    </select>
    <select id="filter-category">
      <option value="all">All Categories</option>
    </select>
  </div>
  <ul class="transaction-list" id="transaction-list"></ul>
</div>
```

### CSS (member3.css)

```
.btn-expense { background-color: #ef4444; }
.btn-expense:hover { opacity: 0.9; }
.filters-container {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1rem;
}
.transaction-list {
  list-style: none;
  max-height: 300px;
  overflow-y: auto;
}
.transaction-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.75rem;
  border-bottom: 1px solid #f3f4f6;
}
.transaction-item.income-type { border-left: 4px solid #10b981; }
.transaction-item.expense-type { border-left: 4px solid #ef4444; }
.transaction-details.tx-cat {
  font-size: 0.75rem;
  color: #6b7280;
  display: block;
}
.delete-btn {
  background: none;
  border: none;
  color: #ef4444;
  cursor: pointer;
  width: auto;
  padding: 0 0.5rem;
}
```

## JavaScript (member3.js)

### JavaScript

```
document.getElementById('exp-date').valueAsDate = new Date();

// Handle Expense Submission
document.getElementById('expense-form').addEventListener('submit', function(e) {
  e.preventDefault();
  const amount = parseFloat(document.getElementById('exp-amount').value);
  const category = document.getElementById('exp-category').value;
  const date = document.getElementById('exp-date').value;

  transactions.push({
    id: Date.now(),
    type: 'expense',
    amount,
    category,
    date
  });

  saveToLocalStorage();
  this.reset();
  document.getElementById('exp-date').valueAsDate = new Date();
});

// Delete item
function deleteTransaction(id) {
  transactions = transactions.filter(tx => tx.id !== id);
  saveToLocalStorage();
}

// Dynamically update category options depending on selected type filter
const typeFilter = document.getElementById('filter-type');
const catFilter = document.getElementById('filter-category');

typeFilter.addEventListener('change', () => {
  updateCategoryOptions();
  updateApp();
});
catFilter.addEventListener('change', updateApp);

function updateCategoryOptions() {
  const type = typeFilter.value;
  catFilter.innerHTML = '<option value="all">All Categories</option>';

  let categories = [];
  if (type === 'all' || type === 'income') categories.push('Salary', 'Freelance',
'Investments', 'Other Income');
  if (type === 'all' || type === 'expense') categories.push('Food & Dining', 'Rent
& Utilities', 'Entertainment', 'Transportation', 'Shopping');

  [...new Set(categories)].forEach(cat => {
    catFilter.innerHTML += `<option value="${cat}">${cat}</option>`;
  });
}

function renderHistory(filteredTransactions) {
  const list = document.getElementById('transaction-list');
  list.innerHTML = '';
```

```

    if (filteredTransactions.length === 0) {
      list.innerHTML = '<li class="transaction-item" style="color:#6b7280">No
transactions found.</li>';
      return;
    }

    const sorted = [...filteredTransactions].sort((a,b) => b.id - a.id);

    sorted.forEach(tx => {
      const li = document.createElement('li');
      li.className = `transaction-item ${tx.type}-type`;
      const prefix = tx.type === 'income' ? '+' : '-';

      li.innerHTML = `
        <div class="transaction-details">
          <strong>${tx.category}</strong>
          <span class="tx-cat">${tx.date}</span>
        </div>
        <div>
          <span style="font-weight:600; color: ${tx.type === 'income' ?
'#10b981' : '#ef4444'}">
            ${prefix}${tx.amount.toFixed(2)}
          </span>
          <button class="delete-btn"
onclick="deleteTransaction(${tx.id})">&times;</button>
        </div>
      `;
      list.appendChild(li);
    });
  }
}

```

#### Member 4: Local Storage, Monthly Reports & CSS Charts

Member 4 manages data persistence using `localStorage`, date grouping logic, dynamic UI re-rendering, and bar chart visuals.

HTML (member4.html)

HTML

```

<div class="panel">
  <div class="report-flex">
    <h2>Monthly Breakdown</h2>
    <select id="report-month-filter" style="width: auto; padding:
0.4rem;"></select>
  </div>
  <div class="chart-container" id="chart-section"></div>
</div>

```

## CSS (member4.css)

```
.report-flex {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1rem;
}
.chart-container { margin-top: 1rem; }
.bar-row { margin-bottom: 0.5rem; }
.bar-label {
  font-size: 0.85rem;
  display: flex;
  justify-content: space-between;
  margin-bottom: 0.25rem;
}
.bar-outer {
  background: #e5e7eb;
  border-radius: 4px;
  height: 12px;
  width: 100%;
  overflow: hidden;
}
.bar-inner {
  height: 100%;
  width: 0%;
  transition: width 0.5s ease-out;
}
.bar-inner.inc-bar { background-color: #10b981; }
.bar-inner.exp-bar { background-color: #ef4444; }
```

## JavaScript (member4.js)

### JavaScript

```
// LocalStorage global state initialization
let transactions = JSON.parse(localStorage.getItem('transactions')) || [];

function saveToLocalStorage() {
  localStorage.setItem('transactions', JSON.stringify(transactions));
  updateApp();
}

const monthFilter = document.getElementById('report-month-filter');
monthFilter.addEventListener('change', updateApp);

function populateMonthFilter() {
  const currentSelection = monthFilter.value;
  monthFilter.innerHTML = '<option value="all">All Time</option>';

  const monthsSeen = new Set();
  transactions.forEach(tx => monthsSeen.add(tx.date.substring(0, 7)));

  Array.from(monthsSeen).sort().reverse().forEach(m => {
    monthFilter.innerHTML += `<option value="${m}">${m}</option>`;
  });

  if (currentSelection) monthFilter.value = currentSelection;
}

function renderReports(filteredTransactions) {
  const chartSection = document.getElementById('chart-section');
  chartSection.innerHTML = '';

  const catTotals = {};
  let maxVal = 0;

  filteredTransactions.forEach(tx => {
    catTotals[tx.category] = (catTotals[tx.category] || 0) + tx.amount;
    if (catTotals[tx.category] > maxVal) maxVal = catTotals[tx.category];
  });

  if (Object.keys(catTotals).length === 0) {
    chartSection.innerHTML = '<p style="color: #6b7280; font-size: 0.9rem;">No data available.</p>';
    return;
  }

  for (const [cat, total] of Object.entries(catTotals)) {
    const isIncome = ['Salary', 'Freelance', 'Investments', 'Other Income'].includes(cat);
    const barClass = isIncome ? 'inc-bar' : 'exp-bar';
    const percentage = maxVal > 0 ? (total / maxVal) * 100 : 0;

    const row = document.createElement('div');
    row.className = 'bar-row';
    row.innerHTML = `
      <div class="bar-label">
        <span>${cat}</span>
        <strong>${total.toFixed(2)}</strong>
      </div>
      <div class="bar-outer">

```

```

        <div class="bar-inner ${barClass}" style="width:
${percentage}%"></div>
    </div>
    `;
    chartSection.appendChild(row);
}
}

// Master Function to update all modules simultaneously
function updateApp() {
    const selectedType = document.getElementById('filter-type').value;
    const selectedCat = document.getElementById('filter-category').value;
    const selectedMonth = monthFilter.value;

    let filtered = transactions.filter(tx => {
        const matchesType = selectedType === 'all' || tx.type === selectedType;
        const matchesCat = selectedCat === 'all' || tx.category === selectedCat;
        const matchesMonth = selectedMonth === 'all' ||
tx.date.startsWith(selectedMonth);
        return matchesType && matchesCat && matchesMonth;
    });

    populateMonthFilter();
    updateDashboard(filtered);
    renderHistory(filtered);
    renderReports(filtered);
}

// App Initialization call
updateCategoryOptions();
updateApp();

```

## Instructions to Combine the Parts

Follow these steps to merge everyone's code into one working project file named `index.html`.

### Step 1: Create the Main HTML Structure

Create a new file named `index.html`. Add the basic template structure with layout grid elements.

#### HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Expense Tracker</title>
  <style>
    /* General Layout CSS */
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: sans-serif; }
    body { background: #f3f4f6; color: #1f2937; padding: 2rem; }
    .container { max-width: 1200px; margin: 0 auto; }
    .main-layout { display: grid; grid-template-columns: 1fr 1fr; gap: 2rem; }
    .panel { background: #ffffff; padding: 1.5rem; border-radius: 12px; margin-
bottom: 2rem; box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1); }
    .panel h2 { margin-bottom: 1rem; font-size: 1.3rem; border-bottom: 2px solid
#f3f4f6; padding-bottom: 0.5rem; }
    @media (max-width: 768px) { .main-layout { grid-template-columns: 1fr; } }

    /* PASTE CSS HERE (Step 3) */
  </style>
</head>
<body>
  <div class="container">
    <header style="text-align:center; margin-bottom: 2rem;">
      <h1>Expense Tracker</h1>
    </header>

    <div class="main-layout">
      <div>
        </div>
      <div>
        </div>
    </div>
  </div>

  <script>
    // PASTE JS HERE (Step 4)
  </script>
</body>
</html>
```

## Step 2: Combine HTML

Paste each member's HTML code into the designated comment blocks in the `index.html` file created above:

1. Place **Member 1 HTML** directly above `<div class="main-layout">`.
2. Place **Member 2 HTML** inside the first column `<div>`.
3. Place **Member 3 Expense Form HTML** inside the first column `<div>`, below Member 2's section.
4. Place **Member 3 History/Filter HTML** inside the second column `<div>`.
5. Place **Member 4 HTML** inside the second column `<div>`, below Member 3's section.

## Step 3: Combine CSS

Paste the CSS snippets from **Member 1, Member 2, Member 3, and Member 4** inside the single `<style>` block in the head, below the general layout styles.

## Step 4: Combine JavaScript

Paste the JS scripts into the single `<script>` block at the bottom of the body in the following **strict order** so functions are loaded before they are called:

1. **Member 4 JS Variables & Save logic:** (`transactions, saveToLocalStorage, populateMonthFilter, renderReports, updateApp`)
2. **Member 1 JS:** (`updateDashboard`)
3. **Member 2 JS:** (`Income Submit Listener`)
4. **Member 3 JS:** (`Expense Submit Listener, deleteTransaction, updateCategoryOptions, renderHistory`)
5. **App Initialization:** Call `updateCategoryOptions()` and `updateApp()` at the very end of the script tag.